

Methodological Support Report v2

Methods, Justifications, and Architectural Interpretation

ANE Spectral Sensing Platform

Document type:	Technical-methodological support report
Scope:	Interface and coordination layers of the platform
Evidence date:	Repository snapshot analyzed on May 23, 2026
Analytical basis:	Context documents, diagrams, service descriptions, data evidence, static inspection
Excluded from direct scope:	Planning artifacts, deep RF DSP internals, independent post-processing internals

Prepared through architectural recovery, documentary triangulation, and methodological synthesis

Abstract

This report provides a second, more methodological version of the architectural analysis for the ANE spectral sensing platform. Its purpose is to explain how the platform is organized, why its principal design choices are reasonable, and which operational rules support monitoring, campaigns, alerts, and compliance reporting, without centering the discussion on file structures or implementation paths. The analysis is based on contextual platform descriptions, frontend and backend flow diagrams, service and database documentation, and static inspection of the implemented solution. The document adopts an evidence-based architectural recovery approach: concerns are identified, system capabilities are grouped into viewpoints, and the resulting strategies are justified according to operational continuity, temporal coherence, maintainability, traceability, and security. The result is a report oriented toward methods, design rationale, and governance rather than code inventory.

Contents

1	Introduction	3
1.1	Problematic Introduction	3
1.2	Purpose	3
1.3	Scope and Delimitation	3
2	Methodological Framework	4
2.1	Methodological Position	4
2.2	Evidence Base	4
2.3	Analytical Procedure	4
2.4	Evaluation Criteria	5
3	System Foundation and Architectural Context	5
3.1	Domain Mission	5
3.2	Operational Boundary	5
3.3	Core Architectural Principle	6
3.4	Cross-Cutting Data Logic	6
4	Frontend Methodological Analysis	6
4.1	Frontend Scope	6
4.2	Interaction Model	6
4.3	Frontend Capability Map	7
4.4	Frontend Design Choices and Their Justification	7
4.4.1	Authentication Before Operational Access	7
4.4.2	Shared Operational Context Across Screens	7
4.4.3	Hybrid Real-Time Interaction	8
4.4.4	Progressive Transition from Monitoring to Campaigns	8
4.4.5	Alert Fusion Instead of Snapshot-Only Monitoring	8
4.5	Frontend Technical Families and Their Justification	8
4.6	Frontend Procedural Rules and Their Justification	9
5	Backend Methodological Analysis	9
5.1	Backend Scope	9
5.2	Architectural Style	10

5.3	Backend Capability Map	10
5.4	Persistence Model and Data Governance	11
5.5	Backend Design Choices and Their Justification	11
5.5.1	Backend as Source of Truth for Monitoring State	11
5.5.2	Hybrid Identity Model	11
5.5.3	Capability-Oriented Service Separation	11
5.5.4	Application-Level Temporal Automation	11
5.5.5	Separated Audio Treatment	12
5.5.6	Report Orchestration Through External Analytical Support	12
5.6	Backend Technical Families and Their Justification	12
5.7	Backend Procedural Rules and Their Justification	12
6	Cross-Cutting Architectural Interpretation	13
6.1	Observed Strengths	13
6.2	Observed Constraints and Design Tensions	14
7	Synthesis of Methods and Justifications	14
7.1	Why the Main Strategies Are Methodologically Coherent	14
7.2	Why the Technical Families Fit the Domain	14
7.3	Why the Operational Rules Matter	15
8	Conclusion	15
A	Evidence Categories Considered	15
B	Analytical Lenses Used	15

1 Introduction

1.1 Problematic Introduction

The platform studied in this repository addresses a complex operational problem: it must transform heterogeneous radio-frequency measurements produced by distributed field sensors into information that can be used for supervision, decision making, and compliance-oriented interpretation. This transformation is demanding because sensing activity is asynchronous, geographically distributed, and exposed to interruptions, stale data, environmental stress, and clock-related inconsistencies. At the same time, the human operators of the platform need both immediate situational awareness and delayed analytical workflows, including scheduled campaigns, alert review, and structured reporting.

Under these conditions, a merely descriptive inventory is insufficient. A methodological report must explain the operational logic of the system, the responsibilities assigned to the interface and coordination layers, the principles that justify their organization, and the procedural rules that give the platform predictable behavior over time. This second version is written with that objective in mind.

1.2 Purpose

The purpose of this document is to provide a methodological support report for the frontend and backend layers of ANE Spectral Sensing Platform. Specifically, it aims to:

- define the operational problem that the platform solves;
- reconstruct the architectural logic of the current implementation from available evidence;
- explain the principal methods used to coordinate monitoring, campaigns, alerts, and reporting;
- justify the main design decisions according to quality attributes and domain constraints; and
- offer a reusable reference for technical reporting, governance, onboarding, and future re-design.

1.3 Scope and Delimitation

This report focuses on the platform layers that mediate between human operators, field sensors, stored information, and analytical services.

- **Included:** access control, operational navigation, live monitoring, campaign management, alert handling, system configuration, service coordination, persistence strategy, temporal automation, and reporting orchestration.
- **Contextually considered:** the field sensing environment, the data-processing boundary, and the analytical post-processing stage. These are examined as dependencies and interfaces, not as the primary subject of the report.
- **Excluded:** planning-only materials, low-level signal-processing internals, and standalone analytical implementation details beyond what is necessary to explain platform behavior.

2 Methodological Framework

2.1 Methodological Position

This report adopts a descriptive and reconstructive software engineering methodology. It is descriptive because it documents the platform as it currently operates rather than as an idealized target design. It is reconstructive because the architecture is inferred from multiple forms of evidence, including domain descriptions, interaction diagrams, service documentation, data-model records, and static inspection of the implementation. The methodological stance is concern-oriented: the report is organized around the questions that matter for system understanding, namely access, monitoring, campaigns, alerts, reporting, persistence, temporal control, and maintainability.

2.2 Evidence Base

The analysis draws on five complementary evidence classes.

Evidence class	Observed content	Methodological contribution
Platform context	General descriptions of mission, actors, work-flows, and system responsibilities	Provides the domain frame that explains why the platform exists and what operational outcomes it must support.
Interaction and coordination diagrams	Narrative flows for the interface layer and the service layer	Clarify the sequence of decisions, the transfer of information, and the boundary between human actions and backend control.
Service and reporting descriptions	Descriptions of access, management, monitoring, campaigns, alerts, and report generation	Reveal the intended service taxonomy and the principal end-to-end use cases.
Database evidence	Tables, constraints, relationships, and time-series notes	Confirms the persistence model, identity anchors, temporal data strategy, and relational consistency assumptions.
Implementation inspection	Static review of representative frontend and backend behaviors	Verifies that the documented responsibilities correspond to actual operational mechanisms.

2.3 Analytical Procedure

The analytical procedure followed four successive stages:

1. **Documentary triangulation:** contextual descriptions, diagrams, and service summaries were compared to extract common vocabulary, system boundaries, and declared responsibilities.
2. **Implementation verification:** the implemented frontend and backend behavior was inspected to confirm how those responsibilities are operationalized.

3. **Architectural synthesis:** the recovered behaviors were grouped into coherent capabilities such as access control, device monitoring, campaign orchestration, alert fusion, reporting, and persistence governance.
4. **Methodological justification:** the resulting design choices were interpreted through quality criteria including temporal coherence, maintainability, traceability, robustness, and security.

2.4 Evaluation Criteria

Architectural decisions are interpreted through the following quality criteria:

- **Functional adequacy:** whether the solution supports the platform’s operational responsibilities.
- **Temporal coherence:** whether the design handles real-time change, scheduled processes, and stale-state risk in a controlled way.
- **Traceability:** whether it is possible to relate observable system behavior to identifiable platform capabilities.
- **Maintainability:** whether the decomposition supports controlled evolution and understandable reasoning.
- **Operational robustness:** whether the solution tolerates disconnections, partial data loss, and long-running workflows.
- **Security posture:** whether identity, authorization, and role boundaries are treated explicitly and consistently enough to support institutional use.

3 System Foundation and Architectural Context

3.1 Domain Mission

The domain mission of the platform is to convert raw spectrum observations into operationally meaningful knowledge. In practice, this means connecting people, remote sensors, stored histories, and analytical services so that the organization can observe the radio environment, respond to incidents, schedule measurement programs, and interpret findings under a compliance-oriented perspective.

3.2 Operational Boundary

The repository evidence reveals a four-part operational environment:

- **Field sensing environment:** distributed devices that measure the radio spectrum, publish state and location, and receive measurement instructions.
- **Interface layer:** the user-facing environment through which operators authenticate, navigate, configure measurements, observe live behavior, manage campaigns, review alerts, and consult reports.
- **Coordination layer:** the service layer that stores information, enforces operational rules, synchronizes users and devices, and orchestrates reporting.

- **Analytical support environment:** external services or processes used for location enrichment and normative interpretation of collected measurements.

3.3 Core Architectural Principle

The dominant architectural principle is the separation between *interaction* and *authoritative control*. The interface layer organizes human workflows and presents data in an interpretable way, but the coordination layer owns the lasting operational state of sensors, campaigns, configurations, and reports. This principle is methodologically appropriate because monitoring systems must continue to behave coherently even when browsers disconnect, operators change screens, or multiple actors interact with the same sensing resources.

3.4 Cross-Cutting Data Logic

The data evidence points to three important structural facts:

- device identity is anchored by a stable hardware identifier that acts as the practical reference across telemetry, configuration, and scheduling;
- campaign and reporting contexts combine normalized relational data with flexible configuration or result payloads, allowing both queryability and controlled variability; and
- time-dependent information is treated as a first-class concern, with explicit attention to historical telemetry, real-time status, and specialized storage patterns for time-series workloads.

4 Frontend Methodological Analysis

4.1 Frontend Scope

The interface layer is responsible for the human experience of the platform. Its mission is not limited to presentation; it structures the sequence through which a person enters the system, understands the current network situation, selects a sensor, initiates live observation, transitions from exploratory monitoring to scheduled campaigns, reviews incidents, and interprets results.

4.2 Interaction Model

The recovered interaction model follows a task-centered sequence:

1. authenticate and validate session state before entering the operational workspace;
2. obtain a general view of the network and device situation;
3. navigate toward live monitoring, campaigns, alerts, or administration according to the task;
4. configure and launch a measurement when direct observation is needed;
5. observe live outputs such as spectrum, location, state, and audio when available;
6. convert a useful measurement configuration into a reusable campaign when continuity is required; and
7. interpret outcomes through historical review, alert examination, and report consultation.

4.3 Frontend Capability Map

The interface layer can be understood through functional capabilities rather than implementation modules.

Capability	Primary concern	Methodological role
Access and session control	Identity validation and protected entry	Ensures that operational screens are reached only after session state is coherent and permission checks are meaningful.
Operational overview	General situation awareness	Gives the user an initial mental model of network state, locations, and activity before more detailed actions are taken.
Device situational awareness	Sensor status, location, and association context	Supports informed sensor selection by exposing the condition and placement of field devices.
Live monitoring configuration	Immediate measurement preparation	Converts operational intent into a concrete acquisition setup that can be executed and later reused.
Spectral interpretation workspace	Real-time visualization and interaction	Makes dense technical measurements readable through charts, markers, units, and contextual controls.
Campaign management	Scheduled measurement planning and review	Bridges one-time observation and repeatable measurement programs governed by time.
Alert review	Incident awareness across current and past behavior	Prevents the operator from relying only on an instantaneous snapshot by preserving historical operational memory.
Administrative configuration	Resource and parameter governance	Keeps the platform adaptable by allowing managed change over devices, users, and general rules.
Audio-assisted observation	Supplemental live interpretation	Extends monitoring beyond visual traces when demodulated audio contributes to interpretation.

4.4 Frontend Design Choices and Their Justification

4.4.1 Authentication Before Operational Access

The interface gives priority to session resolution before the user enters the main workspace. This choice is methodologically justified because it reduces ambiguity at the point where operational decisions begin. A monitoring interface should not oscillate between authenticated and unauthenticated states while the user is already evaluating sensors or measurements.

4.4.2 Shared Operational Context Across Screens

The platform maintains shared context around the currently selected device, measurement state, and active workflow. This is an appropriate design for a control-oriented environment because

monitoring, configuration, alerts, and reporting are not isolated activities; they are different viewpoints over the same operational object.

4.4.3 Hybrid Real-Time Interaction

The monitoring experience combines event-style immediacy with periodic data refresh. This is methodologically coherent because not all payloads behave the same way. State changes and short notifications benefit from immediate delivery, whereas dense measurement arrays are better managed through controlled refresh cycles that protect interpretability and reduce synchronization complexity.

4.4.4 Progressive Transition from Monitoring to Campaigns

The interface allows a live configuration to become the seed of a scheduled activity. This is not a minor convenience; it is a method for preserving continuity between exploratory work and formal measurement plans. It reduces the cognitive and operational cost of translating a useful observation into a repeatable campaign.

4.4.5 Alert Fusion Instead of Snapshot-Only Monitoring

The platform does not rely only on the current state of a device. It combines present conditions with historical alert memory, which is methodologically stronger for operational supervision. A clean instantaneous reading does not erase earlier instability, and a historical alert without current context does not fully describe ongoing risk.

4.5 Frontend Technical Families and Their Justification

The interface layer relies on a family of architectural techniques whose value is methodological rather than merely technological.

Technical family	Observed role	Justification
Component-based interface composition	Organization of screens and operational widgets	Supports complex workflows through reusable units without losing task-level clarity.
Typed interaction contracts	Explicit data structures between view logic and services	Reduces ambiguity in measurement, campaign, identity, and alert payloads.
Geospatial visualization	Device and campaign location awareness	Aligns naturally with field operations in which place is part of the interpretation.
High-density charting	Spectrum and historical trace interpretation	Makes measurement data readable at the scale required by technical supervision.
Stateful orchestration	Coordination of selected sensor, live acquisition, and workflow transitions	Preserves continuity across screens that share the same operational context.

Technical family	Observed role	Justification
Responsive operational layout	Support for dashboards, panels, and side-by-side inspection	Improves usability for long-lived sessions in which comparison and rapid navigation matter.

4.6 Frontend Procedural Rules and Their Justification

Several frontend behaviors are better interpreted as explicit procedures rather than generic interface features.

Rule or procedure	Operational role	Justification
Obsolescence control for live responses	Prevents delayed data from replacing the most recent user context	Necessary when operators switch devices or measurement ranges during active refresh cycles.
Trace reset and duplicate suppression	Keeps waterfall-style histories coherent after configuration changes	Prevents mixed histories and false visual continuity after the measured band changes.
Nearest-point interpretation for markers	Supports precise reading of discrete plotted samples	Appropriate when measurements are visualized as ordered bins rather than continuous analog curves.
Unit conversion for operator interpretation	Presents measurements in multiple meaningful forms	Helps technical users move between instrument-oriented and reporting-oriented views.
Local filtering and progressive review of campaign lists	Supports manageable inspection of moderate result sets	Improves usability without demanding unnecessary complexity in every query path.
Combined live-plus-historical issue evaluation	Merges present device condition with retained incident memory	Produces a more reliable operational picture than either mode on its own.

5 Backend Methodological Analysis

5.1 Backend Scope

The coordination layer is the operational authority of the platform. Its responsibilities include validating access, receiving requests from the interface, accepting telemetry from sensors, storing state and history, coordinating live monitoring, managing campaigns, detecting alerts, communicating relevant updates in real time, and orchestrating report generation through external analytical support.

5.2 Architectural Style

The backend follows a concentrated coordination style rather than a highly fragmented service decomposition. This is methodologically appropriate for the domain because the hardest problem is not independent computation in many separate domains, but the consistent synchronization of users, sensors, measurements, schedules, and reports. A more distributed internal architecture would add communication overhead without necessarily improving the control of shared operational state.

5.3 Backend Capability Map

The coordination layer can be reconstructed through the following capabilities.

Capability	Primary concern	Methodological role
Access and authorization governance	Identity, session, and role control	Separates institutional trust, platform session continuity, and privilege enforcement.
Resource administration	Management of sensors, antennas, users, and parameters	Maintains the controlled environment in which measurements can be executed reliably.
Monitoring orchestration	Live acquisition control and active configuration management	Ensures that a measurement is not just started, but also tracked, bounded, and recoverable.
Campaign orchestration	Scheduled measurement programs	Converts isolated observations into time-governed, repeatable operations.
Telemetry reception	Intake of device state, location, spectrum, and audio	Acts as the ingestion point through which the field environment becomes platform knowledge.
Persistence governance	Historical storage, current-state access, and data consistency	Preserves both transactional entities and temporal evidence required for reporting and audit.
Real-time dissemination	Immediate update propagation	Keeps operators informed without forcing every workflow into manual refresh.
Alert generation	Conversion of abnormal conditions into actionable records	Transforms raw telemetry into operationally meaningful incidents.
Reporting orchestration	Integration of measurements, location context, and analytical interpretation	Produces structured results that can support compliance-oriented reasoning.
Background automation	Periodic validation and lifecycle enforcement	Maintains temporal correctness even when no user is actively supervising a process.

5.4 Persistence Model and Data Governance

The persistence strategy is hybrid. Stable domain entities such as people, devices, physical accessories, and campaigns are represented in relational form because they require integrity, identity, and controlled change. Variable configuration envelopes and analytical results use flexible structures because the domain includes evolving parameters and report payloads that do not fit a rigid schema equally well at all times. Historical telemetry is preserved as chronological evidence, which is consistent with a system where time, recurrence, and traceability are core concerns rather than secondary features.

This strategy is methodologically sound because the platform must serve two kinds of needs at once: durable transactional governance and high-volume temporal observation. The evidence also suggests a design under evolution, especially in the treatment of time-series information and timestamp representations. That is not a methodological flaw by itself, but it introduces an architectural obligation: all time-sensitive logic must explicitly account for representational differences so that schedules, freshness checks, and report windows remain consistent.

5.5 Backend Design Choices and Their Justification

5.5.1 Backend as Source of Truth for Monitoring State

The monitoring lifecycle is maintained as authoritative backend state rather than transient browser state. This is a strong architectural choice because sensing operations can outlive a single user session or screen. If the control layer did not own active monitoring state, the system would be vulnerable to accidental interruption whenever the interface disappears or connectivity degrades.

5.5.2 Hybrid Identity Model

The access design combines institutional identity assurance with a platform-specific authorization envelope. Methodologically, this balances two needs: trust in an external identity source and local control over roles, session lifetime, and application-specific permissions. It also reduces friction by aligning platform access with organizational identity practices.

5.5.3 Capability-Oriented Service Separation

The backend is organized around domain capabilities such as access, measurement control, campaigns, alerts, and reporting. This separation is justified because it preserves conceptual traceability. Even if some capabilities are operationally dense, the domain language remains visible, which is valuable for governance and future refactoring.

5.5.4 Application-Level Temporal Automation

The platform contains explicit time-driven rules for status decay, monitoring timeouts, and campaign transitions. This is methodologically appropriate because temporal behavior in this domain is not incidental; it is part of the business logic. Encoding those rules explicitly in the coordination layer makes them inspectable and auditable.

5.5.5 Separated Audio Treatment

Audio is treated as a distinct monitoring stream rather than as a trivial extension of generic event traffic. This is justified because audio has different framing, continuity, and listener expectations than scalar state updates. A specialized treatment reduces interference between ordinary control messages and higher-throughput interpretive media.

5.5.6 Report Orchestration Through External Analytical Support

The backend acts as the orchestrator of reporting rather than as the sole analytical engine. This is methodologically coherent because reporting in this domain depends on combining stored measurements with contextual enrichment and specialized analysis. A coordinating role allows the platform to integrate these steps while keeping the core platform centered on supervision and control.

5.6 Backend Technical Families and Their Justification

The backend employs several technical families that align with the domain problem.

Technical family	Observed role	Justification
HTTP service coordination	Human-facing requests and configuration operations	Appropriate for administrative and consultative workflows that require explicit request-response semantics.
Event-based real-time messaging	Fast propagation of state changes and live updates	Reduces delay for operational awareness without overloading ordinary query channels.
Relational persistence with flexible payload support	Storage of governed entities plus variable configurations and reports	Balances integrity, queryability, and adaptability.
Time-series aware storage strategy	Historical telemetry capture at operational scale	Matches the need to preserve chronological evidence for monitoring and reporting.
Federated identity plus local session control	Access assurance and privilege enforcement	Supports institutional trust while retaining platform autonomy over authorization.
External analytical integration	Reporting enrichment and interpretation	Keeps specialized analysis available without dissolving the platform's coordinating role.

5.7 Backend Procedural Rules and Their Justification

The backend contains deterministic operational rules that are central to platform reliability.

Rule or procedure	Operational role	Justification
Sensor status decay state model	Classifies device availability according to recent activity	Necessary to distinguish active devices from delayed or disconnected ones in a meaningful way.
Threshold-based alerting with repetition control	Converts abnormal telemetry into actionable incidents without flooding operators	Supports vigilance while preserving the practical value of alert history.
Backend-enforced monitoring timeout	Stops stale or over-extended monitoring sessions	Protects resources and preserves operational discipline when interfaces disappear or sessions are abandoned.
Conflict detection for campaign assignment	Prevents overlapping scheduled use of the same sensing resource	Maintains exclusivity and avoids contradictory instructions to field devices.
Timezone-aware campaign transitions	Evaluates schedule changes in the intended operational timezone	Reduces ambiguity in date-time logic and aligns execution with the real operating calendar.
Configuration-to-device reconstruction rules	Converts stored measurement intent into device-usable parameters	Preserves compatibility when different configuration shapes must still lead to executable measurement behavior.
Threshold-sensitive cache reuse in reporting	Reuses prior analytical results only when the request is semantically compatible	Balances efficiency with correctness in report generation.
Priority-based location resolution	Chooses the best available location source for report context	Improves resilience when one source of geographic information is incomplete or missing.
Specialized audio frame handling	Preserves usable live audio for interpretation	Supports real-time auditory analysis without reducing the clarity of general platform messaging.

6 Cross-Cutting Architectural Interpretation

6.1 Observed Strengths

From a methodological standpoint, the platform shows several strong characteristics.

- **Clear operational decomposition:** the platform distinguishes access, monitoring, campaigns, alerts, administration, and reporting as recognizable concerns.
- **Strong control-plane orientation:** lasting operational state is governed centrally rather

than being left to volatile user sessions.

- **Explicit temporal reasoning:** status windows, schedules, monitoring bounds, and historical traces are treated as design primitives.
- **Interpretability of operational rules:** many important behaviors are deterministic, auditable, and suitable for institutional supervision.
- **Continuity between exploration and formal measurement:** live observation can mature into scheduled campaigns and later into structured reporting.

6.2 Observed Constraints and Design Tensions

The evidence also reveals tensions that matter for future governance.

- **Concentrated orchestration:** some responsibilities remain dense and centrally accumulated, which helps visibility but may increase long-term maintenance effort.
- **Mixed temporal representations:** differing timestamp conventions complicate reasoning across storage, scheduling, and reporting boundaries.
- **Evolutionary time-series strategy:** the persistence approach reflects ongoing refinement rather than a final stabilized model for all telemetry classes.
- **Security normalization pressure:** the access model is conceptually sound, but architectural maturity depends on making enforcement consistently uniform across all sensitive operations.
- **Dependence on external analytical availability:** reporting quality depends not only on stored data, but also on the availability of supporting enrichment and analysis services.

7 Synthesis of Methods and Justifications

7.1 Why the Main Strategies Are Methodologically Coherent

The core strategies of the platform align well with the problem it is solving. A backend-centered control plane is justified because devices, campaigns, and reports persist beyond any one user session. A hybrid real-time model is justified because not all monitoring payloads require the same transport semantics. Explicit temporal automation is justified because the domain depends on scheduled and freshness-sensitive behavior, not only on manual user action. Finally, report orchestration with contextual enrichment is justified because compliance-oriented interpretation demands more than raw measurement retrieval.

7.2 Why the Technical Families Fit the Domain

The selected technical families favor inspectability, controlled integration, and practical evolution. The interface layer uses architectural techniques suited to long-lived, stateful, information-dense workflows. The coordination layer uses patterns suited to governance, persistence, and orchestration. The data layer supports both structured administration and time-dependent evidence. Together, these choices form a pragmatic architecture for a supervision platform rather than an abstract demonstration system.

7.3 Why the Operational Rules Matter

The most important algorithms in the platform are not opaque prediction models; they are explicit operational procedures. Status decay, timeout enforcement, campaign conflict checks, alert deduplication, threshold-aware report reuse, and location prioritization are all understandable rules that can be inspected, audited, and revised. This is methodologically valuable in regulatory and supervisory contexts, where explainability is as important as raw automation.

8 Conclusion

The frontend and backend of ANE Spectral Sensing Platform form a coherent supervision platform whose principal architectural purpose is coordination. The interface layer structures operator activity, guiding users from access control to monitoring, campaigns, alerts, and reports. The backend consolidates authority over measurement state, persistence, timing, and analytical orchestration.

From a methodological perspective, the platform demonstrates a sound alignment between domain needs and architectural choices. Its main strengths are the clarity of its operational concerns, the central role of temporal control, and the use of auditable procedural rules. Its principal tensions involve concentrated orchestration, heterogeneous temporal representation, and the need for progressively stronger normalization of security and time-series strategy. Even so, the evidence supports the conclusion that the platform already constitutes a defensible technical foundation for spectral monitoring, campaign governance, alert supervision, and compliance-oriented interpretation.

This second version of the report therefore serves as a more abstract and method-focused basis for institutional reporting, architectural review, and future redesign because it explains not only what the platform contains, but why its current organization is methodologically reasonable.

A Evidence Categories Considered

- General platform descriptions and mission statements.
- Frontend and backend interaction diagrams.
- Service and workflow documentation for access, monitoring, campaigns, alerts, and reports.
- Database inventory, relationships, constraints, and time-series notes.
- Static inspection of representative interface and coordination behaviors.

B Analytical Lenses Used

- Concern-oriented architectural recovery.
- Workflow reconstruction from user and service flows.
- Persistence and temporal-governance analysis.
- Design-rationale interpretation through quality attributes.
- Cross-cutting assessment of robustness, traceability, and maintainability.